TransitionNode - Transitionカスタムノード

1. 概要

1.1 コンポーネントの目的

TransitionNodeは、ReactFlowで使用されるPetri NetのTransition（遷移）を表すカスタムノードコンポーネントです。矩形形状で、上部に入力ハンドル、下部に出力ハンドルを持ち、Transitionの処理やアクションを表現します。

主要機能:

- Transition情報の視覚的表示（矩形形状）
- ラベル表示
- ステレオタイプ表示（オプション）
- 入力ハンドル（上部）
- 出力ハンドル（下部）
- 選択状態のビジュアルフィードバック
- Ant Design Tokenによるテーマ対応

1.2 使用場面

- PetriNetEditorInnerのReactFlowキャンバス上
- Petri Netの遷移（Method）を表現

1.3 Petri Netにおける役割

- **Transition（遷移）**: 入力Place（Material/Condition）から出力Place（Result）への変換処理を表す
- **意味**: 実験手法やプロセスステップ
- **形状**: 矩形（四角形）

---

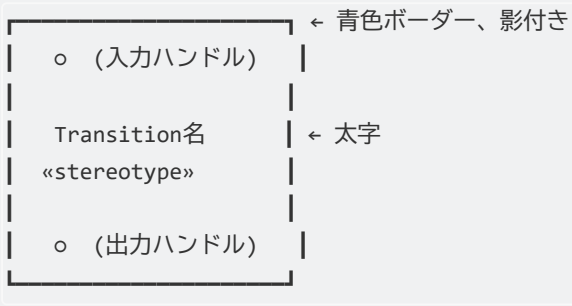
2. 画面構成

2.1 ノードの外観

2.1.1 通常状態



2.1.2 選択状態



2.2 主要UI要素

2.2.1 コンテナ

- **形状**: 矩形（角丸）
- **最小サイズ**: 100px × 60px
- **パディング**: 16px 20px
- **ボーダー**: 2px solid
  - 通常時: token.colorBorder
  - 選択時: token.colorInfo（青色）
- **背景色**: token.colorBgContainer
- **影**: 選択時のみ token.boxShadowSecondary
- **角丸**: token.borderRadiusSM

- **レイアウト**: フレックスボックス（縦方向、中央揃え）
- **トランジション**: all 0.2s

2.2.2 入力ハンドル（上部）

- **位置**: Position.Top
- **type**: target
- **サイズ**: 10px × 10px
- **背景色**: token.colorInfo（青色）
- **ボーダー**: 2px solid token.colorBgContainer

2.2.3 ラベル

- **フォントサイズ**: token.fontSize
- **フォントウェイト**:
  - 通常時: 400
  - 選択時: 600
- **色**: token.colorText
- **配置**: 中央揃え

2.2.4 ステレオタイプ（オプション）

- **表示条件**: data.stereotype が存在する場合
- **フォーマット**: «{stereotype}»
- **フォントサイズ**: token.fontSizeSM
- **色**: token.colorTextSecondary
- **配置**: 中央揃え
- **上マージン**: 4px

2.2.5 出力ハンドル（下部）

- **位置**: Position.Bottom
- **type**: source
- **サイズ**: 10px × 10px

- **背景色**: token.colorInfo（青色）
- **ボーダー**: 2px solid token.colorBgContainer

---

3. 機能仕様

3.1 Props定義

```
interface NodeProps {
  data: {
    label?: string;
    stereotype?: string;
    // ... その他のReactFlow標準プロパティ
  };
  selected: boolean;
  // ... その他のReactFlow標準プロパティ
}
```

Props説明:

Prop名	型	必須	説明
data	object	✅	ノードデータ
data.label	string	-	Transition名（デフォルト: 'Transition'）
data.stereotype	string	-	ステレオタイプ（オプション）
selected	boolean	✅	選択状態フラグ

3.2 ReactFlow統合

3.2.1 ノードタイプ登録

```
const nodeTypes = {
  transition: TransitionNode,
  // ... その他のノードタイプ
};
```

```
<ReactFlow nodeTypes={nodeTypes} ... />
```

3.2.2 ノードデータ構造

```
{
  id: 'transition_1',
  type: 'transition',
  position: { x: 250, y: 100 },
  data: {
    label: 'Synthesis',
    stereotype: 'method',
    // ... その他のデータ
  }
}
```

3.3 スタイル動的変更

3.3.1 選択状態の反映

```
border: `2px solid ${selected ? token.colorInfo : token.colorBorder}`,
boxShadow: selected ? token.boxShadowSecondary : 'none',
fontWeight: selected ? 600 : 400,
```

効果:

- 選択時: 青色ボーダー、影付き、ラベル太字
- 非選択時: 通常ボーダー、影なし、ラベル通常

3.4 ハンドル配置

3.4.1 入力ハンドル（Target）

- **配置**: 上部中央
- **接続方向**: 上からの入力（Place → Transition）
- **ReactFlow Position**: Position.Top

3.4.2 出力ハンドル（Source）

- **配置**: 下部中央
- **接続方向**: 下への出力（Transition → Place）
- **ReactFlow Position**: Position.Bottom

3.5 メモ化

```
const TransitionNode: React.FC<NodeProps> = memo(({ data, selected }) => {
  // ...
});
```

説明: React.memoでラップされ、props変更時のみ再レンダリング

---

4. スタイル仕様

4.1 コンテナスタイル

```
style={{
  padding: '16px 20px',
  borderRadius: token.borderRadiusSM,
  border: `2px solid ${selected ? token.colorInfo : token.colorBorder}`,
  background: token.colorBgContainer,
  boxShadow: selected ? token.boxShadowSecondary : 'none',
  minWidth: 100,
  minHeight: 60,
  display: 'flex',
  flexDirection: 'column',
  justifyContent: 'center',
  alignItems: 'center',
  transition: 'all 0.2s',
}}
```

Ant Design Tokenマッピング:

スタイルプロパティ	Token	説明

borderRadius	borderRadiusSM	小さい角丸
border (選択時)	colorInfo	青色
border (通常時)	colorBorder	グレー
background	colorBgContainer	背景色
boxShadow (選択時)	boxShadowSecondary	第2レベルの影

4.2 ラベルスタイル

```
style={{
  fontSize: token.fontSize,
  fontWeight: selected ? 600 : 400,
  color: token.colorText,
  textAlign: 'center',
}}
```

4.3 ステレオタイプスタイル

```
style={{
  fontSize: token.fontSizeSM,
  color: token.colorTextSecondary,
  textAlign: 'center',
  marginTop: 4,
}}
```

4.4 ハンドルスタイル

```
style={{
  background: token.colorInfo,
  width: 10,
  height: 10,
  border: `2px solid ${token.colorBgContainer}`,
}}
```

---

5. インタラクション

5.1 選択

- **操作**: ノードをクリック
- **ReactFlowが処理**: selected prop が true に変更
- **結果**: ボーダー色が青色、影表示、ラベル太字

5.2 ドラッグ

- **操作**: ノードをドラッグ
- **ReactFlowが処理**: position更新
- **結果**: ノード移動

5.3 接続

- **入力接続**: Place（Material/Condition）からTransitionへ
- **出力接続**: TransitionからPlace（Result）へ
- **ReactFlowが処理**: Edge作成

5.4 ホバー

- **トランジション**: all 0.2s で滑らかな変化

---

6. Petri Net用語との対応

6.1 Petri Net要素

Petri Net用語	MAIMLでの意味	TransitionNodeでの表現
Transition	Method（実験手法）	このノード
Input Place	Material/Condition	入力ハンドル（上部）

Output Place	Result	出力ハンドル（下部）
Arc	データフロー	ReactFlow Edge

6.2 ステレオタイプの使用例

stereotype	意味	使用ケース
method	実験手法	標準的なTransition
process	プロセス	複数ステップを含む処理
analysis	分析	データ分析処理
synthesis	合成	化学合成処理

---

7. 実装ファイル

7.1 ファイルパス

- **メインファイル**: `frontend/src/components/editorv2/components/nodes/TransitionNode.tsx`

7.2 依存ライブラリ

- **reactflow**: Handle, Position, NodeProps
- **antd**: theme (useToken)
- **react**: memo

7.3 親コンポーネント

- PetriNetEditorInner（ReactFlowのnodeTypes経由）

7.4 関連コンポーネント

- MaterialPlaceNode
- ConditionPlaceNode

- ResultPlaceNode

---

8. 使用例

8.1 ReactFlowでのノードタイプ登録

```
import TransitionNode from './components/nodes/TransitionNode';

const nodeTypes = {
  transition: TransitionNode,
  materialPlace: MaterialPlaceNode,
  conditionPlace: ConditionPlaceNode,
  resultPlace: ResultPlaceNode,
};

<ReactFlow
  nodes={nodes}
  edges={edges}
  nodeTypes={nodeTypes}
  onNodesChange={onNodesChange}
  onEdgesChange={onEdgesChange}
  onConnect={onConnect}
/>
```

8.2 ノードデータの作成

```
const newTransitionNode = {
  id: `transition_${Date.now()}`,
  type: 'transition',
  position: { x: 250, y: 150 },
  data: {
    label: 'Synthesis',
    stereotype: 'method',
    methodId: 'method_001',
    properties: { ... }
  }
};

setNodes([...nodes, newTransitionNode]);
```

8.3 ノードの選択

```
// ReactFlowが自動的にselected propを管理
// 手動で選択状態を設定する場合:
setNodes(nodes.map(node => ({
  ...node,
  selected: node.id === selectedNodeId
})));
```

---

9. テーマ対応

9.1 Ant Design Token使用

```
const { token } = useToken();
```

使用されるToken:

- `token.borderRadiusSM`: 角丸サイズ
- `token.colorInfo`: プライマリカラー（青色）
- `token.colorBorder`: ボーダー色
- `token.colorBgContainer`: 背景色
- `token.boxShadowSecondary`: 影
- `token.fontSize`: フォントサイズ
- `token.fontSizeSM`: 小フォントサイズ
- `token.colorText`: テキスト色
- `token.colorTextSecondary`: 副次テキスト色

9.2 テーマ変更への対応

- Ant Designのテーマを変更すると、自動的にノードのスタイルも変更される

- ダークモード対応も自動

---

10. パフォーマンス最適化

10.1 React.memo

```
const TransitionNode: React.FC<NodeProps> = memo(({ data, selected }) => {
  // ...
});
```

効果: props変更時のみ再レンダリング

10.2 トランジションアニメーション

```
transition: 'all 0.2s'
```

効果: 選択状態変更時の滑らかなアニメーション

---

11. アクセシビリティ

11.1 推奨される改善

- ノードにaria-label追加
- キーボード操作対応（ReactFlowのデフォルト機能を利用）
- スクリーンリーダー対応

11.2 実装例

```
<div
  role="button"
```

```
  aria-label={`Transition: ${label}${stereotype ? `, stereotype: ${stereotype}` : ''}`}
  tabIndex={0}
  style={{ ... }}
>
```

---

12. 変更履歴

日付	変更内容	担当者
2026-03-12	実装ベースで仕様書を新規作成	Cline
2026-01-29	Ant Design準拠版（v2.0.0）	Development Team